

Live Check-in Feature Overview

High-Level Overview

The Live Check-in feature allows members of a community to join real-time audio and video calls directly within the TaskTrove application. It is designed to facilitate quick syncs, daily stand-ups, or collaborative sessions related to specific challenges.

When a user clicks "Live Check-in" on a community page, a modal opens, prompts for camera and microphone permissions, and connects them to a shared virtual room. Other members of the community can join the same room to see and hear each other.

Key capabilities include:

- **Real-time Video & Audio:** Powered by WebRTC for low-latency communication.
 - **Multi-Peer Support:** Supports multiple participants in a grid layout.
 - **Media Controls:** Users can intuitively toggle their camera and microphone on or off.
 - **Presence & Participant Count:** Displays the number of active participants and their names/avatars.
 - **No External Servers:** Uses Firebase Firestore for signaling, avoiding the need for a dedicated WebSocket server.
-

Technical In-Depth Overview

The architecture is entirely peer-to-peer (P2P) using WebRTC (Web Real-Time Communication), with Firebase Firestore acting as the signaling server.

1. WebRTC Connection Setup (Signaling)

WebRTC requires a signaling mechanism for peers to exchange connection information (Session Description Protocol - SDP) and network routing paths (ICE Candidates) before a direct P2P connection can be established.

- **Firestore as Signaling Server:** Instead of a traditional Socket.io server, we use Firestore subcollections (`communities/{id}/liveRooms/{roomId}/signals`).
- **Offer/Answer Flow:**
 1. When User A joins, they write a "Participant" document.
 2. When User B joins, the deterministic logic dictates who sends the first SDP "Offer" (handled by comparing `userId` lexicographically to avoid collision races).
 3. User A creates an `RTCPeerConnection`, generates an Offer, and saves it to the `signals` collection.
 4. User B receives the Offer via a Firestore `onSnapshot` listener, applies it (`setRemoteDescription`), generates an "Answer", and writes it back.
 5. As network paths are discovered, "ICE Candidates" are written and exchanged similarly.

2. Media Stream Management

- **Local Stream:** Captured via browser's `navigator.mediaDevices.getUserMedia()`. It's stored in a React state variable (`localStream`) so the UI reacts immediately when the camera turns on.
- **Remote Streams:** When a peer connection resolves, the `ontrack` event fires, providing the remote `MediaStream`. This is mapped to the peer's ID and stored in the React component's state to render `<RemoteVideoTile>`.
- **Dynamic Attachment Fixes:** To handle browser `autoplay` policies, explicitly calling `video.play()` inside `onloadedmetadata` ensures feeds don't freeze when tracks are dynamically toggled or attached.

3. Renegotiation

A critical feature implemented in `LiveCheckinRoomManager` is `onnegotiationneeded`. If a user joins *before* granting camera permissions, or if they add/remove tracks dynamically, WebRTC requires renegotiation (a new Offer/Answer exchange) so the remote peer can update its stream bindings.

4. Component Structure

- `LiveCheckinDialog.tsx`: The main UI. Handles React state, triggers modal opening, asks for permissions, and manages the grid layout (`VideoGrid`, `LocalVideoTile`, `RemoteVideoTile`).
- `liveCheckinWebrtc.ts`: The pure WebRTC business logic layer. Completely decoupled from React, it handles the `RTCPeerConnection` lifecycle, Firestore listeners, cleanup, and heartbeat mechanisms.

Limitations

1. Mesh Network Scaling (N-to-N):

- We are using a Mesh topology. Every user creates a dedicated `RTCPeerConnection` to every other user.
- If 5 users join, each user uploads 4 video streams and downloads 4 video streams.
- **Limitation:** Bandwidth and CPU usage scale exponentially. It becomes unstable beyond ~6-8 participants. For larger meetings, an SFU (Selective Forwarding Unit) server like livekit or mediasoup would be required.

2. Turn Servers:

- Currently, we only configure free Google STUN servers (`stun:stun.l.google.com:19302`). STUN servers just help peers find their public IP.
- **Limitation:** If two users are behind strict enterprise NATs or symmetric firewalls, STUN will fail, and they won't be able to connect. A TURN server (which actually relays the video data) is needed for 100% reliability in restricted network environments.

3. Firestore Read/Write Costs:

- Signaling via Firestore means every ICE candidate and heartbeat generates document reads/writes. While fine for small usage, extremely active rooms might hit Firebase quota limits if not carefully monitored.
-

Future Implementations & Enhancements

1. Screen Sharing:

- Utilizing `navigator.mediaDevices.getDisplayMedia()` to capture the screen and dynamically add it as an additional video track to existing `RTCPeerConnections`.

2. Active Speaker Detection:

- Using Web Audio API to analyze audio track volumes and highlight the border of the tile belonging to the currently speaking user.

3. TURN Server Integration:

- Provisioning a STUN/TURN service like Twilio Global Network (NTS) or Metered TURN to guarantee connection success across all firewalls.

4. Chat (Text) overlay:

- Adding a local chat within the modal that saves messages to a subcollection in the `liveRooms` path, allowing participants to share text or links while on video.

5. Connection Quality Indicators:

- Using WebRTC `getStats()` API to monitor packet loss and display a network quality icon (Green/Yellow/Red) on peer tiles.